1. 总体架构：固定 encoder + distill decoder

1.1 Teacher 设计

为了“先小后大”，建议这样分两层：

- **Phase 1 (Mini)**：
  - Teacher：`whisper-tiny` 或 `tiny.en`
  - 参数 ~40M，HPC/单卡都压得住
- **Phase 2 (Full)**：
  - Teacher：`whisper-small` 或 `whisper-base`
  - 在你 pipeline 跑通后再切换

Teacher 使用方式：

- 只做 inference，不训练
- 功能：
  1. 给原始数据集打 pseudo-label（teacher transcript）
  2. 在 training 时提供每个 token 的 logits 或 softmax 分布，供 KD / DKD 使用

1.2 Student 设计（decoder-only）

你已经决定 student 用 decoder 版本，这是很好的选择。

一个合理的结构是：

- **Encoder**：
  - 直接复用 teacher encoder（tiny/small 的 encoder）
  - **完全冻结**（不更新参数）
- **Decoder (student)**：
  - 同样的 vocab、同样的 positional encoding 形式
  - 但：
    - 减少 layer 数（如从 6 层减到 2/3/4）
    - 或者减少 hidden size / head 数
  - 你可以定义几种 student：
    - S2：2-layer decoder
    - S3：3-layer decoder

        - S4：4-layer decoder
    - 这些 student 用来画 speed–accuracy curve，也顺带满足课程里“不同架构”的对比需求

好处：

- 大头 encoder 直接沿用 teacher 表征，student 只负责语言建模部分
- 算力主要花在 decoder 上，蒸馏训练相对轻很多
- 非常适合做 DKD（因为 DKD本来就是针对分类 logits 的）

1.3 Loss 框架（三个层次）

你可以定义三个层次的 loss，用来一步步升级：

1. **CE-only baseline**
    - 只用 ground-truth transcript 做 cross-entropy
    - 和 Project 6 中“仅用硬标签 student”的角色对应
2. **classical KD**（Hinton）
    - CE + KL(teacher_logits || student_logits) with temperature T
    - 对 decoder 每个 time step 的 vocab 分布做 KL
3. **DKD（TCKD + NCKD）**
    - 把 KL 拆成两部分：
        - TCKD：只看 target token 维度
        - NCKD：只看非 target token 的分布结构
    - 总 loss：

$$L_{\mathrm{DKD}} = \alpha \cdot L_{\mathrm{TCKD}} + \beta \cdot L_{\mathrm{NCKD}} + \gamma \cdot L_{\mathrm{CE}} \tag{1}$$

你按照这个顺序从简单到复杂实现/实验即可。

---

2. Phase 1：最小可跑版本（小 teacher + 小数据）

目标：**一周内能看到 KD 比 CE-only 好一点的实验结果**，证明方案可行。

2.1 数据选取（小而真实）

建议：

- 数据集：LibriSpeech `train-clean-100` 的一个子集，比如：
    - 训练：5–10 小时音频
    - 验证：1 小时

- 数据预处理：
    - 限制句子时长 ≤ 10s
    - 保持标准的 Whisper 特征提取（log-mel spectrogram）

这样，一个小 epoch 非常快（几十分钟甚至更少），便于快速 iterate。

2.2 三个 baseline 模式（只在 Mini setting 里做）

在这一步你只用 tiny 作为 teacher & encoder，student decoder 选一个比较小的（比如 2 层）：

1. **Baseline A：CE-only student**
    - encoder = freeze(teacher.encoder)
    - decoder = S2
    - loss = CE(student_logits, ground-truth tokens)
2. **Baseline B：classical KD student**
    - 同样的 S2 decoder
    - loss = α·CE + (1-α)·KD
    - KD = KL(softmax(z_T / T) || softmax(z_S / T))，在 token 维度上平均
3. **Baseline C：teacher 直接 fine-tune（可选）**
    - encoder + decoder 都 train（或只 decoder）
    - 作为“上限”参考

你在这个阶段不必上 DKD，只要：

- 显示：
    - CE-only vs KD 在 val WER / CER 上有差距（哪怕是 0.5–1%）
    - 或者在 small-data regime 下，KD 收敛更稳、overfitting 更慢

这样你已经满足：

- “student 仅用硬标签 vs 用 soft labels 的性能对比”
- “distillation 能带来 improvement” —— 课程最核心要求之一

2.3 温度 T 的小实验

仍然用 S2 decoder + tiny teacher，在小数据集上试：

- T = 1（等价于没温度）
- T = 2

- T = 4

对比：

- 验证集 WER / CER
- KD loss 收敛情况

这一小实验对应 Project 6 的 “Try different temperature T and analyze influence”。

你可以简单分析：

- 小 T：teacher 分布很“尖锐”，KD 主要强调 top-1 / top-2 token
- 大 T：分布更平滑，student 可以学到更多“相近 token”的相对关系

---

3. Phase 2：标准 KD 完整版（更大 teacher + 更大数据）

在 Phase 1 跑通之后，你可以安全放大：

3.1 放大内容

1. **Teacher 换成 whisper-small / base**
    - encoder & decoder 都更强，logits质量更好
    - 你可以继续复用 tiny 做“Mini teacher”，small 做“Full teacher”比较
2. **数据量增加**
    - 从 5–10 小时扩展到 50–100 小时（train-clean-100，甚至 + train-clean-360 的子集）
    - 但你不必用到全部 460 小时，保持在你机器能承受的范围内
3. **Student decoder 变成 S2 / S3 / S4 三种配置**
    - 用 classical KD 在每种 decoder 深度上训练
    - 拿到一个速度–性能曲线（类似 Project 6 中不同架构对比）

3.2 这一步的 deliverables（对应课程要求）

在这个阶段，你已经可以产出很多符合课程打分点的结果：

- 每个模型的：
    - 参数量
    - FLOPs（可以估算）
    - RTF（real-time factor）
    - WER / CER
- 实验：

- CE-only vs classical KD
- 不同 T（在 small setting 基本完成，full setting 可以只选最优 T）
- 不同 decoder depth（2/3/4）

这些结果已经 **至少不输"在 CIFAR 上玩 KD 的卷积项目"** 了，甚至更有亮点。

---

4. Phase 3：DKD 设计 —— 如何在 decoder token 上实现 TCKD / NCKD

这是你项目的"魂"。

4.1 基本思想回顾

在 DKD 里，每个样本/类别的 loss 被拆成：

- **Target-Class KD (TCKD)**：只关心 true label 那一维上，teacher vs student 的概率一致性
- **Non-Target-Class KD (NCKD)**：只针对其他类别的分布结构，对应"类间结构"的知识

迁移到 ASR 上，就是把"类别"换成"当前 time step 的正确 token"。

4.2 在 decoder 上的具体形式

对于每个 time step t：

- teacher logits：$z_t^T \in \mathbb{R}^V$
- student logits：$z_t^S \in \mathbb{R}^V$
- ground-truth token：$y_t \in 1, \ldots, V$

先做 softmax with temperature T：

$$p_t^T = \mathrm{softmax}(z_t^T/T), \quad p_t^S = \mathrm{softmax}(z_t^S/T) \tag{2}$$

然后：

1. **TCKD 部分**（只看 y_t 那一维）：

$$L_{\mathrm{TCKD},t} = -, p_t^T(y_t) \log p_t^S(y_t) \tag{3}$$

这相当于让 student 在"正确 token"上的概率尽量接近 teacher。

2. **NCKD 部分**（对其他 token 做 KL）：

- 把 y_t 这维去掉/掩掉，只在其余维度上做 KL：

$$\tilde{p}_t^T = \frac{p_t^T(k), k \neq y_t}{\sum_{k \neq y_t} p_t^T(k)} \tag{4}$$

$$\tilde{p}_t^S = \frac{p_t^S(k), k \neq y_t}{\sum_{k \neq y_t} p_t^S(k)} \tag{5}$$

$$L_{\mathrm{NCKD},t} = \mathrm{KL}(\tilde{p}_t^T | \tilde{p}_t^S) \tag{6}$$

3. **整体 DKD loss**（对所有 time steps 加权平均）：

$$L_{\mathrm{DKD}} = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \left(\alpha L_{\mathrm{TCKD},t} + \beta L_{\mathrm{NCKD},t} \right) \tag{7}$$

4. **总训练目标**：

$$L_{\mathrm{total}} = \gamma \cdot L_{\mathrm{CE}} + L_{\mathrm{DKD}} \tag{8}$$

你在实验中可以：

- 固定 γ = 1（CE 权重），扫描 α, β
- 或者把 α+β 固定，总体权重再乘一个 λ

4.3 DKD 的 ablation 计划（对标论文 Table 1）

在 CIFAR 里，他们做的是：

- CE
- CE + KD
- CE + TCKD only
- CE + NCKD only
- CE + DKD（TCKD + NCKD）

你可以在 decoder 上做完全对应的一套：

对一个固定的设置（比如 teacher=small, student=S3, 数据集固定）：

1. **CE-only**
2. **CE + classical KD（统一 KL）**
3. **CE + TCKD only（β=0）**
4. **CE + NCKD only（α=0）**
5. **CE + DKD（α, β > 0）**

观察：

- WER / CER：哪种最好?
- 收敛曲线：哪种最稳定?
- 是否出现“仅 NCKD > 仅 TCKD”的现象 —— 这是 DKD 论文的关键发现之一

这一步基本上就把 Project 6 的最后两 bullet **完整搬到语音领域**了。

---

5. Phase 4：从“小版 DKD 实验”到“更大规模”

为了控制时间+成本，你可以先在“中等规模”上做 DKD，然后再决定要不要放大到更大 teacher 或更多数据。

一个合理顺序：

1. **Mini DKD**：
    - teacher = whisper-tiny
    - student = S2
    - 数据：5–10 小时
    - 只跑几 epoch，看 NCKD vs TCKD 的趋势
2. **Medium DKD**：
    - teacher = whisper-small
    - student = S3 / S4
    - 数据：50–100 小时
    - 做完整 ablation（CE, KD, TCKD, NCKD, DKD）
3. **Optional Large DKD**：
    - teacher = whisper-base
    - student = S3
    - 数据：>100 小时
    - 只跑 classical KD vs DKD，对 final result 做重点展示

你可以在报告里说明：

> 由于算力限制，我们在 small teacher + 50h 数据上做了最完整的 ablation，而在 base teacher + 100h 数据上做了 classical KD vs DKD 的高配对比。

老师会认为这是非常合理的 tradeoff。

---

6. 整体时间线建议（对应你 proposal 的时间表）

结合你现在的目标和课程要求，我建议把“实验节奏”规划成：

- **Week 1–2**
    - 搭好数据 pipeline 和 Whisper 特征提取
    - 跑通 CE-only student（tiny encoder + S2 decoder）
    - 在小数据集上出一版 baseline WER

- **Week 3**
  - 实现 classical KD，跑 Mini KD 实验（tiny + S2 + 5–10h）
  - 做 T=1,2,4 的小对比
  - 初步展示 CE-only vs KD 的 improvement
- **Week 4**
  - 换 teacher = whisper-small
  - 数据扩到 50–100 小时
  - 在 S2/S3/S4 上跑 classical KD，收集 speed–accuracy 曲线
  - 这一步已经能支撑一个“中期检查 / midterm poster”
- **Week 5**
  - 实现 DKD（TCKD + NCKD）
  - 在 small teacher + S3 上做完整 ablation（CE/KD/TCKD/NCKD/DKD）
  - 分析：NCKD 是否主导性能提升，DKD 是否优于 classical KD
- **Week 6**
  - 视 compute 再决定是否做 base teacher / 更多数据
  - 汇总：
    - 曲线图（WER–FLOPs / decoder depth）
    - 不同 loss 的对比表格
    - 温度 T 的影响
    - CE-only vs KD vs DKD 对比
  - 写 final report + poster

这样你既保证：

- 有 **minimally working project**（很早就见到 improvement）
- 又能在后期做出 **有研究味道的 DKD vs KD 对比**